FirstKnock Pipeline Audit Report

FirstKnock Pipeline Audit Report

Full Audit: Steps 1–3, Chunking Logic & Redfin Discrepancy Root Cause

April 7, 2026

Prepared by	Senior Backend Engineer — Pipeline Audit
Date	April 7, 2026
Classification	Internal Technical Audit
Scope	Full audit of the FirstKnock data pipeline covering Steps 1–3, chunking logic, and Redfin discrepancy root cause

Executive Summary

This audit covers the full FirstKnock data pipeline: Step 1 (/properties endpoint ingestion), Step 2 (local heuristic classification), Step 3 (BatchData validation), the 300 sq mile chunking architecture, and the Redfin 'Pending' discrepancy. The audit identifies two confirmed critical bugs in Step 2, a root-cause cost bleed in Step 3, and architectural gaps in the chunking and classification logic. Remediation priorities are ordered by impact.

Four critical findings require immediate action before the next production deployment. Two are silent failures — they produce no errors but generate incorrect classifications at runtime.

2P0 Silent Failures (statusCategory, soldPrice)	**~215**Records/Area Sent to BatchData (Should Be ~21)	**~92%**Potential BatchData Cost Reduction	**4**Sections Audited

Critical Findings

1. statusCategory field does NOT exist in the RentCast API — any code referencing it silently returns undefined.
2. soldPrice field does NOT exist on any RentCast listing endpoint — only lastSalePrice on

/properties exists.

3. Step 3 (BatchData) is receiving ~215 records/area when it should receive ~21 — a 10times cost overrun caused by upstream filter failure.
4. The Redfin 'Pending' discrepancy is a Step 2 classification error, not a data source conflict.

Section 1 — Step 1 Audit (/properties Endpoint)

saleDateRange Parameter

Verdict: CONFIRMED CORRECT at saleDateRange=180.

The research documentation confirms that saleDateRange=180 is required — not 90 — to reliably capture homes closed within the last 90 days. The reason is county deed recording lag: after a property closes, the county deed is typically recorded 30–90 days later. A saleDateRange=90 window will miss homes whose deeds were recorded late (e.g., a home that closed 85 days ago but whose deed was only recorded 10 days ago would not appear in a 90-day window). Using 180 days ensures the full 90-day closing window is captured even accounting for maximum recording lag.

Risk: If any code path uses saleDateRange=90, recent sales will be silently missed. This must be audited across all call sites.

Pagination

Verdict: CONFIRMED CORRECT.

Correct pattern: limit=500, offset increments by 500 per page, terminate when response array length < 500 or response is an empty array []. This matches the documented RentCast pagination behavior.

Deduplication

Use formattedAddress as the primary deduplication key (or native property id if present). Upsert logic: on conflict, compare lastSaleDate — update the record only if the incoming lastSaleDate is newer than the stored value. This prevents stale data from overwriting fresher records.

Fields to Extract

The following fields must be extracted on every record:

Table 1 — Required Fields for Extraction on Every /properties Record

Field	Purpose
lastSaleDate	Primary sale confirmation signal
lastSalePrice	Used for non-arm's length filtering
owner.names	Used for corporate/institutional flagging
ownerOccupied	Occupancy status
propertyType	Property classification
formattedAddress	Primary deduplication key
id	Secondary deduplication key

Non-Arm's Length Filter

Discard any record where lastSalePrice is $0, $10, or $100. These price points are characteristic of family trust transfers, LLC transfers, and intra-family conveyances — they are not market-rate sales and should not be treated as door-knocking leads.

Corporate/Institutional Flag

Flag any record where owner.names contains any of the following strings (case-insensitive): LLC, Trust, Bank, Inc, Holdings, Properties. Route flagged records to the investor workflow rather than the residential door-knocking pipeline.

Section 1 Verdict

Step 1 is architecturally correct if and only if saleDateRange=180 is used. The non-arm's length filter and corporate flag logic are sound. The primary risk is any code path that uses saleDateRange=90.

Section 2 — Step 2 Audit (Local Heuristic Classification)

CRITICAL BUG 1: statusCategory Does Not Exist in RentCast

Severity: Critical. Silent failure. No error is thrown — the logic evaluates against undefined and produces incorrect classifications.

The field statusCategory does not exist anywhere in the RentCast API. It is absent from /listings/sale, /properties, and every other RentCast endpoint. Any code that references statusCategory will silently receive undefined at runtime — no error is thrown, no exception is raised, the logic simply evaluates against undefined and produces incorrect classifications.

The status field on /listings/sale is binary only: the only valid values are 'Active' and 'Inactive'.

There are no Sold, Pending, Expired, or Withdrawn values in the RentCast API. Any classification logic that attempts to read statusCategory to distinguish between these states is broken.

Table 2 — Field Existence Summary: Critical Classification Fields

Field	Exists in RentCast?	Valid Values	Notes
statusCategory	NO — Confirmed Absent	N/A	Absent from all endpoints. Any reference returns undefined silently.
status	YES	Active, Inactive	Binary only. No granular disposition values exist.
soldPrice	NO — Confirmed Absent	N/A	Only price (list price) exists on listing records.
lastSalePrice	YES — /properties only	Numeric (deed-recorded)	Carries 30–90 day deed recording lag.

Action required: Audit every .ts file for any reference to statusCategory. Replace with the heuristic scoring model described below.

CRITICAL BUG 2: soldPrice Does Not Exist in RentCast

Severity: Critical. Silent failure. Any code that reads soldPrice from a listing record is reading undefined.

The field soldPrice does not exist on any RentCast listing endpoint. The /listings/sale endpoint exposes only price (the list price). Confirmed sale price is only available as lastSalePrice on the /properties endpoint, and it carries a 30–90 day deed recording lag.

Any code that reads soldPrice from a listing record is reading undefined. This means any logic that uses soldPrice to confirm a sale is silently failing.

Action required: Remove all references to soldPrice from listing-based logic. Use lastSalePrice from /properties only, with the understanding that it reflects the deed-recorded price with lag.

Heuristic Scoring Model

Per the research documentation, a heuristic scoring model is proposed for classifying Inactive listings. Implementation status in the .ts files could not be confirmed (files could not be parsed). The model uses the following fields: daysOnMarket, removedDate, listedDate, lastSeenDate, and the history object.

Table 3 — Heuristic Scoring Model: Signal Weights

Signal	Field(s) Used	Condition	Score
Fast removal — sold indicator	daysOnMarket	< 30 days	+3
Short duration — sold indicator	removedDate - listedDate	< 45 days	+2
Gradual removal — sold pattern	lastSeenDate vs. removedDate	Gap geq 7 days before removedDate	+1
First-time listing — sold indicator	history object	Single entry only	+1
Extended market time — expired indicator	daysOnMarket	> 150 days	-3
DOM boundary clustering — expired indicator	daysOnMarket	approx 90, 180, or 365 days (plusminus3 days)	-2
Duration boundary clustering — expired indicator	removedDate - listedDate	approx 90, 180, or 365 days (plusminus3 days)	-2
Abrupt removal — withdrawn indicator	lastSeenDate vs. removedDate	Same day as removedDate (gap = 0)	-2
Ultra-short listing — withdrawn indicator	removedDate - listedDate	< 7 days	-1
Repeat listing — failed attempts indicator	history object	3 or more prior entries	-1

Table 4 — Classification Decision Thresholds

Total Score	Classification	Action	Estimated Confidence
geq 3	LIKELY SOLD	Classify as Sold — no BatchData call required	75–85%
1 or 2	PROBABLY SOLD	Classify as Sold with lower confidence flag	60–70%
-1 to 0	AMBIGUOUS	Route to BatchData for authoritative classification	N/A
-2 or -3	PROBABLY EXPIRED	Classify as Expired with lower confidence flag	60–70%

leq -4	LIKELY EXPIRED / WITHDRAWN	Classify as Expired or Withdrawn — no BatchData call required	75–85%

history Object Limitations

The history object exists on /listings/sale (added September 2024) but records only 'Sale Listing' or 'Rental Listing' events — it does not record outcomes such as Sold, Expired, or Withdrawn. The history object cannot be used to determine why a listing was removed from active status. It can only be used as a signal for listing churn (multiple prior listing events suggest a property that has been relisted repeatedly, which is a negative sold-signal).

Lag Window Rule

If removedDate is older than 90 days AND no Phase 1 /properties match exists in the database rightarrow auto-classify as EXPIRED/WITHDRAWN. This rule is proposed in the false alarms documentation but implementation status requires verification.

Delta Validation

Maintain a persistent store of previously seen listingId values. On each fetch cycle, only process NEW listingId values not seen in the previous cycle. This is expected to reduce the processing pool from ~215 records/week to ~13 new records/week — a ~94% reduction. This optimization is proposed but implementation status requires verification.

Section 2 Verdict

Step 2 is the primary source of BatchData cost bleed. The statusCategory and soldPrice bugs mean the classification logic is broken at its foundation. The heuristic model and Lag Window Rule need to be implemented and verified. Until Step 2 is fixed, Step 3 will continue to receive the full unfiltered Inactive pool.

Section 3 — Step 3 Audit (BatchData Validation)

Current Cost Profile

$0.10 Cost Per Record	**~215**Records Sent Per Area (Current)	**$172/mo**Monthly Cost Per Rep	**~$13/mo**Target Monthly Cost Per Rep

Table 5 — BatchData Cost Profile: Current vs. Optimized

Metric	Current State	Optimized Target	Reduction
Records sent per area	~215 (full Inactive pool)	~21 (AMBIGUOUS only)	~90%
Cost per area	~$21.50	~$1.68	~92%
Monthly cost per rep (8 areas)	~$172	~$13.44	~92%
BatchData Growth tier break-even	47 areas/month	200 areas/month (25 reps)	N/A

Root Cause of Cost Overrun

The root cause is upstream filter failure in Steps 1 and 2. Because Step 2 is not filtering aggressively enough (due to the statusCategory and soldPrice bugs, and the absence of the Lag Window Rule and Delta Validation), the full ~215 Inactive pool is being forwarded to BatchData. The correct architecture sends only AMBIGUOUS records (heuristic score -1 to 0) to BatchData — estimated at ~10% of the total pool, or ~21 records per area.

Async Bulk Endpoint

The async bulk endpoint (POST https://api.batchdata.com/api/v1/property/lookup/async) is proposed in the false alarms documentation but is not confirmed as implemented. The current implementation may be making synchronous 1-by-1 API calls, which is both slower and more expensive per-call than the bulk endpoint.

Section 3 Verdict

The root cause is upstream filter failure. Fix Steps 1 and 2 first. Then migrate to the async bulk endpoint. Do not attempt to optimize Step 3 in isolation — the cost savings come from reducing the input volume, not from changing the BatchData call pattern.

Section 4 — 300 Sq Mile Chunking Audit (processFetchChunk)

A single circle covering 300 sq miles has radius r = " (3 0 0 / π) approx 9.77 miles. However, a single circle cannot cover a square or irregular 300 sq mile area without leaving gaps at the corners. The correct approach is a grid of overlapping sub-circles.

For a 300 sq mile rectangular area (approximately 17.3 times 17.3 miles), using sub-circles of radius 5 miles (area approx 78.5 sq mi each), a 4times4 grid of 16 sub-circles with ~20% overlap provides full coverage with no gaps.

Cross-Chunk Deduplication — CRITICAL

Sub-circle overlap means the same property will appear in multiple chunk results. Cross-chunk deduplication is mandatory. Failure to deduplicate will result in inflated record counts and duplicate BatchData calls.

Deduplicate by formattedAddress or property id before merging results from all sub-circles. Failure to deduplicate will result in inflated record counts and duplicate BatchData calls.

Job Tracking Metrics

The metrics total_fetched, total_inserted, and total_api_calls must aggregate across all sub-circles, not just the last one. If these counters are reset or overwritten per sub-circle, the final job metrics will be incorrect.

Error Handling

Each sub-circle fetch must be wrapped in an independent try/catch block. A failed sub-circle should log the error and continue processing remaining sub-circles — do not abort the entire job on a single sub-circle failure. Track failed_chunks as a separate metric. This ensures partial coverage is better than no coverage.

Coverage Verification

After all chunks complete, verify coverage by checking that the bounding box of returned properties spans the expected geographic area. This catches cases where a sub-circle returned zero results due to a silent API error.

Section 4 Verdict

The chunking approach is architecturally sound but requires: (a) correct overlap math verified against the target area geometry, (b) mandatory cross-chunk deduplication, (c) per-chunk error isolation with failed_chunks tracking, and (d) accurate aggregate metrics across all sub-circles.

Root Cause

Redfin sources listing status from MLS syndication feeds. When a property goes under contract, the listing agent marks it Pending in the MLS. This status syndicates to Redfin in near real-time (typically within hours).

The FirstKnock pipeline sources from county deed records via /properties. A deed is only recorded after closing — typically 30–90 days after the contract is signed.

This creates a structural timing gap: a property can simultaneously be Pending on Redfin (contract signed, not yet closed) while appearing in the FirstKnock pipeline as a sold lead if the pipeline is using /listings/sale Inactive heuristics rather than confirmed deed records.

Table 6 — Data Source Timing Comparison: Redfin vs. FirstKnock Pipeline

Data Source	Update Trigger	Latency	Confirms Sale?
Redfin (MLS syndication)	Listing agent marks Pending in MLS	Near real-time (hours)	NO — contract signed, not yet closed
FirstKnock /properties (deed records)	County clerk records the deed	30–90 days after closing	YES — legally confirmed transfer
/listings/sale Inactive heuristics	Property removed from active MLS	Real-time	NO — includes Pending, Expired, Withdrawn, Cancelled

Classification Error

This is a Step 2 classification error. The heuristic model is classifying Pending listings (under contract, not yet closed) as sold because they were removed from active MLS status. A listing goes Inactive on RentCast when it is removed from active MLS — this happens both when a property sells AND when it goes under contract (Pending). The heuristic model cannot distinguish between these two cases without additional signals.

Fix

The pipeline should not classify a property as sold until either:

1. Phase 1 /properties returns a lastSaleDate for the property, OR
2. BatchData confirms the sale via its own deed/public records lookup.

Properties classified by heuristics only should be flagged as PROBABLY SOLD — PENDING CONFIRMATION and held out of the door-knocking queue until confirmed.

Decision Rule

If a property shows Inactive on RentCast but Pending on a Redfin cross-check, hold it in a pending_confirmation queue and re-check in 30 days. At that point, either the deed will have been recorded (Phase 1 will confirm it) or it will have been on the market long enough to classify as expired/withdrawn.

Remediation Priority Order

The following table orders all identified issues by priority. P0 items must be resolved before any other work proceeds. P1 items drive the majority of cost reduction. P2 and P3 items improve reliability and lead quality.

Table 7 — Remediation Priority Order

Priority	Issue	Impact	Effort
P0	Remove all statusCategory references	Fixes silent classification failure	Low
P0	Remove all soldPrice references from listing logic	Fixes silent classification failure	Low
P1	Implement Lag Window Rule (90-day auto-reject)	Reduces BatchData input ~40%	Medium
P1	Implement Delta Validation (listingId deduplication)	Reduces BatchData input ~94%	Medium
P1	Implement heuristic scoring model	Reduces BatchData input to ~10%	High
P2	Migrate BatchData to async bulk endpoint	Reduces per-call overhead	Medium
P2	Add cross-chunk deduplication in processFetchChunk	Fixes duplicate records	Low
P3	Add Redfin cross-check for Pending confirmation	Reduces false positives	High

Estimated Pipeline Outcomes After Full Remediation

Table 8 — Estimated Pipeline Outcomes After Full Remediation

Pipeline Stage	Resolution Method	Est. % of Inactive Records	BatchData Required?
Hard signal — /properties deed lookup	Step 1 cross-reference confirms lastSaleDate	~35%	No
Heuristic scoring — high confidence	Score geq 3 (LIKELY SOLD) or leq -4 (LIKELY EXPIRED)	~55%	No
Ambiguous — routed to BatchData	Heuristic score -1 to 0	~10%	Yes
Lag Window auto-reject	removedDate > 90 days, no Phase 1 match	Subset of above	No

~35%Resolved by /properties Hard Signal	**~55%**Resolved by Heuristic Scoring	**~10%**Routed to BatchData (Ambiguous Only)	**$5–15/mo**Est. BatchData Cost (Down from $500+/mo)

End of FirstKnock Pipeline Audit Report